

AXON-RFC-006: Stateful EasyNet

Minimal Normative Core — the protocol primitives a stateful agent network requires

Silan Hu

National University of Singapore

Draft v0.1, April 28, 2026

Abstract

This document defines the minimal protocol surface a stateful EasyNet deployment must implement. It deliberately limits itself to seven sections covering the primitives that Phase 1 implementation work requires — state object identity, canonical state hashing, receipt class separation, transition ability schema, authority and delegation, view projection, and receipt indexing by state key — and explicitly defers everything else (full theorems, hub deployment, web-app and pages reference machines) to companion artefacts. The companion document `AXON-RFC-006-stateful-easynet.md` contains a pressure-test appendix (Appendix B, `easynet.web_app`) whose thirteen invariants this main body is engineered to satisfy. The relationship between the two documents is intentional: the appendix was written *first*, as the hardest concrete state object the protocol must keep consistent; this main body then commits the minimal vocabulary and rules under which the appendix’s invariants hold. Section 8 lists a coverage matrix proving that every appendix invariant has at least one normative anchor here, and that every locally-introduced invariant in this document (SO/CSH/RC/TA/VP-INV-*) is self-contained. The work this document directly authorises is Phase 1: schema-level `canonical:bool` marking, the two-class receipt split, `principal/` caller URIs, a content-as-blob boundary, and a state-key-keyed receipt index.

Contents

1	Reading guide	2
2	State Object	3
2.1	Definition	3
2.2	State type	3
2.3	State key	3
2.4	Lifecycle	3
2.5	Identity immutability	4
2.6	Relationship to abilities	4
2.7	Schema-level canonicity marking	4
3	Canonical State Hash	4
3.1	Three-step computation	4
3.2	CSH-INV-1: hash over projection, not over raw state	5
3.3	CSH-INV-2: content-as-blob boundary	5
3.4	Hash and version binding	6
4	Receipt Classes	6
4.1	CanonicalReceipt	6
4.2	OperationalReceipt	6
4.3	No mixing	6
4.4	RC-INV-1: receipt binding	7

4.5	Long-running transition pattern	7
4.6	Caller-context channel	7
4.7	Concurrent transition admissibility	8
5	Transition Ability	8
5.1	transition_id	8
5.2	Transition schema fields	8
5.3	TA-INV-1: receipt-to-transition binding	9
5.4	Validation timing	9
5.5	Cross-state-space composition	9
6	Authority Domain and Delegation	9
6.1	Owner agent	10
6.2	Owner signature on canonical receipts	10
6.3	Delegated execution	10
6.4	The principal/ caller URI scheme	11
7	View Projection	11
7.1	Definition	11
7.2	Multi-view as first-class	11
7.3	agent_view minimum content	12
7.4	VP-INV-1: ability_surface is derived, not registry-snapshot	12
7.5	Determinism	12
8	Receipt Indexing by State Key	12
8.1	Dual indexing requirement	12
8.2	URA mapping	13
8.3	Receipt-history enumeration via URA	13
8.4	Append-only	13
9	Invariant Coverage Matrix	13
9.1	Appendix B invariants → main-body anchors	14
9.2	Main-body invariants (self-contained)	14
9.3	Coverage statement	14
10	Phase 1 implementation work plan	14
11	Closing	15

1 Reading guide

Every substantive statement in this document carries one of three tags, identical in role to the tags used in the Axon Invocation Axiom paper:

[normative] A foundational rule of the stateful protocol. Binding on every Phase 1 implementation. Revising a **[normative]** statement requires a separate alignment round and invalidates every **[derived]** statement downstream of it.

[derived] Logically derived from one or more **[normative]** statements. Negotiable *only* by showing the derivation is incorrect or that the rule it depends on has been revised.

[deferred] A gap deliberately left open. Each **[deferred]** point names the artifact (companion paper, future RFC, or Appendix B sub-section) that will resolve it.

Four typographic boxes appear:

- A red **Invariant** box pins the binding commitment of a section in one sentence.
- A blue **Note** box adds clarification or pointers to related documents.
- An orange **Transitional misalignment** box names a specific place where the current EasyNet-CLI implementation contradicts the rule. These are debts, tracked explicitly, to be discharged in Phase 1.
- A purple **Open decision** box marks a choice that must be resolved before implementation proceeds.

Reading order. If reading for the first time, read §2 through §8 in order; the dependency graph is acyclic. §9 (the coverage matrix) is best read *after* the rest, with the companion appendix open in a second window.

2 State Object

2.1 Definition

[normative]

Definition 1 (State object). A *state object* is a triple

$$\mathcal{S} = (t, k, v(\tau))$$

where t is a *state type*, k is a *state key* unique within t , and $v(\tau)$ is the value of the object at logical time τ , evolving under a sequence of *transitions*. The pair (t, k) is the object's identity; v is its content.

2.2 State type

[normative] A *state type* t is a stable string identifier with the syntactic shape `<namespace>.<noun>`, both lowercase ASCII. Examples: `easynet.page`, `easynet.web_app`. A state type names a *class* of state objects sharing one schema, one transition vocabulary, one canonical projection, and one view declaration set. Two implementations claiming to host the same state type **MUST** agree on all four byte-for-byte.

2.3 State key

[normative] A *state key* k is a tuple of typed scalars whose tuple schema is fixed by the state type. The state key uniquely identifies one instance within t . State key fields **MUST** include enough information to derive the canonical authority for the object (§6); placing the `owner_agent` URI inside the state key is the standard mechanism, because authority that lived as a mutable field could be rewritten by a transition, opening a privilege-escalation primitive the protocol refuses to expose.

2.4 Lifecycle

[derived] A state object's lifecycle is the sequence

$$\emptyset \xrightarrow{\text{first canonical transition}} v_1 \rightarrow v_2 \rightarrow \cdots \rightarrow v_n \rightarrow (\text{terminal})$$

Each v_i is the canonical value of the object after the i -th canonical transition; transitions are defined in §5. The terminal state is type-specific (e.g. `Deleted` for many object types); after the terminal, further transitions **MUST** be rejected.

2.5 Identity immutability

Invariant — SO-INV-1

The state key of a state object **MUST NOT** change after the first `CanonicalReceipt` for that key has been emitted. A state object cannot be renamed, re-keyed, or moved across types; the only way to “rename” is to delete one object and create a distinct one.

Rationale. The audit chain in §8 indexes receipts by state key. A mutable key would break receipt enumeration: receipts emitted before the key change would no longer be discoverable through the new key. Implementations are expected to enforce SO-INV-1 at the receipt-validation layer (a canonical transition that would mutate any state-key field is rejected before apply).

2.6 Relationship to abilities

[derived] A state object is *not* an ability. Abilities are the verbs that mutate a state object; the object itself is the noun. One ability **MAY** affect zero or more state objects (per its declared transition schema, §5); one state object is acted on by one or more abilities. The protocol does not permit an ability to invent its own state object types at runtime — every transition references a state type known statically through schema registration.

2.7 Schema-level canonicity marking

[normative] The schema of every state type **MUST** annotate every field with an explicit Boolean flag `canonical`. Fields with `canonical:true` participate in `canonical_state_hash` (§3); fields with `canonical:false` do not. There is no implicit default; omitting the flag is a schema error.

Note

The Phase 1 implementation in `EasyNet-Cli` *does not yet* carry this annotation on `AbilityDescriptor` or on state-object schemas. The schema upgrade and the validator that refuses unannotated schemas are the first two work items of Phase 1a. See §9 for the appendix invariants this section anchors.

3 Canonical State Hash

3.1 Three-step computation

[normative] The `canonical_state_hash` of a state object is computed in three deterministic, sequentially-composed steps:

1. **Canonical projection.** A pure function

$$\pi : v(\tau) \mapsto v_{\text{can}}(\tau)$$

that selects the subset of fields with `canonical:true` (§2.7) and discards everything else. The projection **MUST** be schema-driven; an implementation that hand-rolls the projection for a specific state type produces a hash incompatible with peer implementations.

2. Canonical serialization. A pure function

$$\sigma : v_{\text{can}} \mapsto b \in \{0,1\}^*$$

producing a deterministic byte string. The serialization **MUST** be RFC 8785 (JSON Canonicalization Scheme, JCS): every implementation hashing the same canonical projection produces the same bytes regardless of language, map iteration order, or floating-point printer.

3. Hashing. A pure function

$$h : b \mapsto \text{canonical_state_hash}$$

using a single fixed cryptographic hash (Phase 1: SHA-256; upgradeable via a schema-level hash version field, but not swappable per call).

Open decision

Open: future hash agility. SHA-256 is the Phase 1 choice. A migration path to an algorithm with stronger collision margin (SHA3-256 or BLAKE3) is required eventually. Mechanism: a per-state-type schema field `hash_version` with a registry of algorithms. The mechanism is specified in this RFC; the migration policy is deferred to RFC-009.

3.2 CSH-INV-1: hash over projection, not over raw state

Invariant — CSH-INV-1

`canonical_state_hash` **MUST** be computed over $\sigma(\pi(v))$, not over $\sigma(v)$ or any other transformation of v . An implementation that hashes raw state, or that performs its own ad-hoc projection, produces values incompatible with conforming peers and is non-conformant.

Why this invariant exists. Without an explicit projection step, distinct implementations diverge on which fields enter the hash even when their schemas agree — different languages serialize maps in different orders, different SDKs canonicalize floats differently, and over time a field accidentally tagged `canonical:false` in one SDK and `canonical:true` in another produces silently different hashes. CSH-INV-1 forces the projection to be a first-class step, schema-driven, auditable.

3.3 CSH-INV-2: content-as-blob boundary

Invariant — CSH-INV-2

A state object field whose semantically-meaningful value is large (e.g. an HTML body, an image, a build artifact) **MUST** be represented in canonical state as a content hash (a fixed-length byte string), with the actual bytes stored in a content-addressed blob store external to the state object. Embedding the bytes directly in canonical state is non-conformant.

Note

The blob-store interface (`put/get/exists`, ref-counting and garbage-collection rules) is **[deferred]** to RFC-007. Phase 1 may implement a minimal local blob store that satisfies `put/get/exists`; ref-counting and GC may be stubbed.

3.4 Hash and version binding

[derived] The pair (`canonical_state_hash`, `state_version`) together identifies one specific value in the lifecycle of one state object. `state_version` is a monotonic integer equal to the number of `CanonicalReceipts` that have advanced the object's state key (§4). A verifier replaying canonical receipts produces a sequence of pairs with strictly increasing versions; if two distinct hashes are observed at the same version, the receipt log is corrupt.

4 Receipt Classes

4.1 CanonicalReceipt

[normative] A `CanonicalReceipt` witnesses one canonical transition of one state object. Its required fields are:

```
CanonicalReceipt {
  receipt_class:      "canonical",
  state_type:         string,
  state_key:          tuple,
  pre_state_hash:     bytes(32),      // canonical_state_hash
                                   // BEFORE the transition;
                                   // null/zero only for the
                                   // creation transition
  post_state_hash:    bytes(32),      // canonical_state_hash
                                   // AFTER the transition
  state_version:      uint64,
  transition_id:       string,         // §4
  attempt_id:         string,         // §3.4
  owner_signature:    bytes,          // §5.2
  observed_at:        timestamp
}
```

A `CanonicalReceipt` **MUST** be replayable offline: given a sequence of canonical receipts in version order, a verifier reconstructs the same canonical state without consulting any network channel.

4.2 OperationalReceipt

[normative] An `OperationalReceipt` records a runtime or observability event. It **MUST NOT** carry `pre_state_hash` or `post_state_hash` that refer to canonical state; the canonical chain is closed under `CanonicalReceipts` alone. An `OperationalReceipt` **MAY** carry runtime metadata (executor node, pid, port, log references, progress fractions), **MAY** carry the `caller_context` channel (§4.6), and is not required for replay correctness: the loss of any subset of operational receipts **MUST NOT** affect the recoverability of canonical state.

4.3 No mixing

[normative] A single ability invocation **MUST NOT** produce both a `CanonicalReceipt` and an `OperationalReceipt` for the same logical effect. A long-running canonical transition produces many operational progress receipts *plus* exactly one terminal canonical receipt; this is one logical effect spread across receipts of different classes, but each receipt witnesses a distinct event (a progress observation versus the canonical commit) and the classification is unambiguous.

4.4 RC-INV-1: receipt binding

Invariant — RC-INV-1

Every receipt **MUST** carry the triple

$$(\text{state_key}, \text{transition_id}, \text{attempt_id})$$

identifying exactly which transition attempt the receipt belongs to. **attempt_id** **MUST** be unique within a **(state_key, transition_id)** pair and **MUST** be preserved across all operational and canonical receipts participating in the same attempt.

Why this invariant exists. Without **attempt_id**, two concurrent invocations of the same transition on the same state key cannot be told apart in the receipt log: their progress streams interleave indistinguishably, and a verifier cannot attribute a given progress receipt to a specific canonical terminal. With **attempt_id**, every receipt is unambiguously attributable.

4.5 Long-running transition pattern

[derived] A long-running transition (one whose wall-clock duration spans more than a single network round-trip) emits a receipt sequence of the form

```
on success:
  [1] OperationalReceipt  transition.started
  [2] OperationalReceipt  transition.progress      (0..N, lossy)
  [3] CanonicalReceipt    transition.completed
                           pre_state_hash = hash before attempt
                           post_state_hash = hash after success

on failure:
  [1] OperationalReceipt  transition.started
  [2] OperationalReceipt  transition.progress      (0..N, lossy)
  [3] CanonicalReceipt    transition.failed
                           pre_state_hash = hash before attempt
                           post_state_hash = hash after failure
```

The properties this implies are sharp:

- Canonical state advances at exactly one point per attempt, namely the terminal canonical receipt.
- Replay from **CanonicalReceipts** alone reaches the correct terminal; the transient “in-flight” state is invisible to a verifier without operational receipts.
- Progress receipts are observability, not history. A peer connecting late receives only the canonical terminal; one connecting early sees the progress stream as well.

4.6 Caller-context channel

[normative] An **OperationalReceipt** **MAY** carry a structured **caller_context** map alongside (not replacing) **envelope.caller**. The map records observations the call site made about the request; its presence **MUST NOT** influence **canonical_state_hash**, and its absence **MUST NOT** affect canonical replay correctness. Specific field schema is delegated to the call site (§6.4 specifies one in the principal-URI context). RFC-006 main commits only to the channel’s existence and to its non-canonical nature.

4.7 Concurrent transition admissibility

Invariant — RC-INV-2

Every canonical transition attempt **MUST** declare its expected `pre_state_hash`. A canonical receipt whose declared `pre_state_hash` does not match the current `canonical_state_hash` of its state key **MUST** be rejected at receipt-validation time, before any apply step.

This is optimistic concurrency: two attempts on the same state key may proceed in parallel; whichever's terminal canonical receipt arrives first wins, the other's receipt is rejected (the loser re-reads state and retries). RFC-006 main *commits* to optimistic concurrency rather than strict serialization because the former preserves liveness under network partition: a node that cannot temporarily reach the owner of a state key can still queue attempts whose viability is checked at the owner's next contact. Strict serialization, while simpler to implement, requires a single in-flight transition per key globally and is network-partition-fragile.

Note

RC-INV-2's `pre_state_hash` field is the same field already required in §4. The novelty here is the validation rule: rejection is at the receipt log's gate, not at the apply step. An apply-time check would let two attempts both complete locally and only collide at convergence time, which is not an admissible failure mode.

5 Transition Ability

5.1 transition_id

[normative] A *transition* is an ability whose schema declares it as canonical or operational. Every transition is identified by a stable `transition_id` string of the form

`<ability-qualified-name>@v<version>`

where `<ability-qualified-name>` follows the ordinary EasyNet ability naming scheme (`<namespace>.<verb>`), and `<version>` is a monotonically-increasing integer assigned by the schema registry when the ability's transition schema changes in a non-backward-compatible way.

Examples: `webapp.build@v1`, `pages.publish@v2`.

Schema evolution. A receipt referring to `webapp.build@v1` is forever validated under the `v1` schema; a transition schema upgrade allocates a new `transition_id` (`@v2`). Old receipts replay under their original schema; new attempts use the new `transition_id`. The receipt log is therefore stable under unbounded schema evolution; this is the same discipline an append-only event store applies to schema versioning.

5.2 Transition schema fields

[normative] A transition schema declares, at minimum:

```
{
  transition_id:    string,                // see §4.1
  transition_class: "canonical" | "operational",
  pre_state:       set<state_value>,      // admissible inputs
  post_state:      set<state_value>,      // admissible outputs
}
```



```

args_schema:    JSONSchema,
receipt_schema: JSONSchema           // schema of the
                                     // receipt this
                                     // transition emits
}

```

The `pre_state` and `post_state` sets are interpreted relative to the state type's state space; an admissible attempt is one whose owning object's current state value is in `pre_state`, and whose successful outcome lies in `post_state`.

5.3 TA-INV-1: receipt-to-transition binding

Invariant — TA-INV-1

Every receipt **MUST** reference its transition's `transition_id`. Validators **MUST** resolve that `transition_id` against the schema registry and validate the receipt's payload against the schema's `receipt_schema`. Receipts referencing an unknown `transition_id` **MUST** be rejected.

This is the schema-validation hook for the entire receipt log. Without TA-INV-1, replay must trust that historic receipts conformed to whatever schema was in force at the time, with no way to verify; with TA-INV-1, the schema registry is the source of truth, and the receipt log is auditable against it indefinitely.

5.4 Validation timing

[normative] Transition schema validation **MUST** occur at receipt-validation time (the moment a receipt is admitted to the log), not at apply time. A receipt that fails schema validation **MUST** be rejected before any state mutation; an implementation that allows non-conformant receipts to apply locally and only rejects them at convergence time is non-conformant.

5.5 Cross-state-space composition

[normative] A state type **MAY** declare multiple parallel state spaces on the same state key (the canonical / runtime overlay split of Appendix B is the minimal interesting case; types declaring review, billing, or audit overlays are admissible future cases). When a state type declares $n \geq 2$ state spaces, its schema **MUST** include a *cross-product admissibility matrix* specifying which combinations of values across the spaces are allowed; transitions producing an inadmissible combination **MUST** be rejected at receipt-validation time.

Note

The cross-product matrix is what prevents a runtime crash from being modelled as a canonical state change. With only one state space, an implementation cannot express the difference; with two spaces and an explicit admissibility matrix, the impossibility of (`canonical=Building`, `runtime=Running`) is a schema-level fact, not a runtime invariant the application must remember.

6 Authority Domain and Delegation

6.1 Owner agent

[normative] Every state object has exactly one *owner agent*, identified by an EasyNet agent URI. The owner agent is the *canonical authority* for the object: every **CanonicalReceipt** for the object **MUST** carry the owner agent’s signature (§6.2).

The owner agent’s identity **MUST** be embedded in the state key (§2.3), not in a mutable field. Embedding it in the key turns “who can sign canonical transitions for this object” into a property of the object’s identity rather than a property of its content. Because keys are immutable (SO-INV-1), authority cannot be silently rewritten by a transition; the only way to change ownership is to delete the object and create a distinct one with a different key.

6.2 Owner signature on canonical receipts

[normative] A **CanonicalReceipt** for state key k **MUST** carry a cryptographic signature

$$\text{owner_signature} = \text{Sign}_{\text{sk}_{\text{owner}(k)}}(\sigma(\text{receipt body}))$$

where $\text{sk}_{\text{owner}(k)}$ is the private key bound to the owner agent identity in k , and σ is the canonical serialization of §3. A canonical receipt without owner signature **MUST** be rejected; a receipt whose signature does not verify against the owner agent’s published key **MUST** be rejected; verification **MUST** be performed at receipt-validation time.

Open decision

Open: signature scheme. Phase 1 commits to Ed25519 for its small key size and well-understood security profile. Threshold or multi-signature schemes (for shared-ownership objects, e.g. organisations) are **[deferred]** to RFC-008.

6.3 Delegated execution

[normative] A canonical transition **MAY** be *executed* by a node other than the owner — the source of a webapp may live on agent A , the build farm on agent B , the serve runtime on agent C — but the owner remains the canonical authority. The delegated execution flow is:

1. The owner emits an authorisation token (or signs an attempt-prefix) addressed to the delegate agent and bound to (`state_key`, `transition_id`, `attempt_id`).
2. The delegate executes the transition; on success or failure, it constructs the canonical receipt body but does *not* sign it.
3. The delegate returns the receipt body to the owner.
4. The owner verifies the body, signs it with $\text{sk}_{\text{owner}(k)}$, and emits the signed **CanonicalReceipt**.

A delegate-emitted receipt without owner signature **MUST NOT** advance canonical state. The owner’s signature is the authoritative act; the delegate’s execution is mechanical labour.

Note

The authorisation token format and the round-trip latency budget for the delegate → owner signing handshake are **[deferred]** to RFC-008. Phase 1 may implement a synchronous loop in which the owner signs canonical receipts on demand from co-located delegates.

6.4 The principal/ caller URI scheme

[normative] The `envelope.caller` field of every invocation **MUST** be a URI under one of two namespaces:

- **Agent caller.** `easynet:///agent/<id>` — the caller is an EasyNet agent. This is the original (pre-stateful) form.
- **Principal caller.** `easynet:///principal/<class>/<...>` — the caller is a principal that is not an EasyNet agent. The `<class>` segment is one of a small set of reserved classes (`human-anon`, `human-auth`, `bot-auth`; new classes require an RFC). Subsequent segments are class-specific.

The reserved class `human-anon` has the form

```
easynet:///principal/human-anon/<ip-hash>/<session-id|"-">
```

where `ip-hash` is a stable hash (e.g. `HMAC(local_salt, ip).hex[..16]`) and `session-id` is an opaque token or the literal `"-"` when absent. The class `human-anon` is the conformant placeholder for a hub-translated public browser request; using the hub's own agent URI as caller for such requests is non-conformant.

Disambiguation discipline. Authorisation, scope-check, and audit code paths **MUST** discriminate `agent/` from `principal/` by URI prefix and **MUST NOT** treat a `principal/` caller as carrying agent-level identity guarantees. Specifically: a `principal/human-anon/...` caller has no signature, no key, no realm membership; any code path that relies on those for an `agent/` caller **MUST** fail fast on a `principal/` caller rather than degrading silently.

Transitional misalignment

Phase 1 implementation: the existing `envelope.caller` parser in `EasyNet-Cli` assumes `agent/` form throughout. Acceptance of `principal/` URIs requires extending the parser, extending scope-check, and adding a fail-fast classifier in `ability_proxy`. This is Phase 1c work.

7 View Projection

7.1 Definition

[normative] A *view* is a pure projection of a state object's (canonical state, runtime overlay state) pair, and, optionally, of the `caller_context` of the request that asked for the view. A view computation **MUST NOT** mutate any state, **MUST NOT** consult side channels (network, time of day, load, locale) for its output content, and **MUST** produce identical bytes when given identical inputs.

(View *caching* may consult time. View *content* may not.)

7.2 Multi-view as first-class

[normative] A state type **MAY** declare $n \geq 0$ named views in its schema. Any state type intended for exposure through a Hub-class deployment **MUST** declare at least the two reserved views

- `human_view` — HTTP-renderable (HTML, image, stream).

- **agent_view** — structured, machine-actionable.

The two views **MUST** produce structurally distinct outputs; serving the same byte sequence to a human GET and an agent GET on the URA channel is non-conformant content negotiation, not multi-view.

7.3 agent_view minimum content

[normative] The **agent_view** of any state object **MUST** include, at minimum:

1. A snapshot of the canonical state (the values of all canonical fields at the current version).
2. A snapshot of the runtime overlay state, when the state type declares one.
3. An **ability_surface** listing the **transition_ids** admissible from the current state.
4. A pointer to the receipt-history view of this state object (§8).

It **MAY** additionally include an **eal_hint** (a ready-to-paste EAL invocation that advances the state object) and implementation-specific fields enabled by the state type’s view schema.

7.4 VP-INV-1: ability_surface is derived, not registry-snapshot

Invariant — VP-INV-1

The **ability_surface** of an **agent_view** **MUST** be derived from the current state of the object and the transition schema (specifically: the set of **transition_ids** whose **pre_state** contains the current state value, intersected with the cross-product admissibility matrix of §5.5). It **MUST NOT** be a static snapshot of the agent’s ability registry.

Why this invariant exists. An **ability_surface** that simply lists “every ability the agent has ever registered” defeats state-gating: a peer agent reading the view would attempt calls that are inadmissible from the current state, and the state-gating logic would have to be replicated at every caller. Deriving the surface from current state and schema makes the view a genuine affordance: the listed transitions are precisely those that will succeed if attempted now.

7.5 Determinism

[normative] Every view of a state type is a deterministic function of its declared input set. The default input set is (canonical, runtime); views that consult **caller_context** **MUST** declare it as an explicit input. No view **MAY** consult **caller_context** ambiently or partially; the dependency is either declared and deterministic, or absent.

8 Receipt Indexing by State Key

8.1 Dual indexing requirement

[normative] The receipt log **MUST** support enumeration along two axes simultaneously:

- **By receipt_id** — preserves the pre-stateful “one invocation is traceable end-to-end” property of the original EasyNet protocol.
- **By state_key** — enables the new “one state object’s entire life is traceable end-to-end” property.

A log that supports only one axis is non-conformant.

8.2 URA mapping

[normative] Every state object **MUST** be addressable by a stable URA derived from its state key alone, via a mapping function declared by the state type’s schema. The mapping **MUST** be bijective on $(\text{state_type}, \text{state_key}) \leftrightarrow \text{ura}$ within a realm: two state objects map to the same URA if and only if they have the same key under the same type, and the URA can be inverted to recover both.

The standard form is

`easynet:///r/<realm>/agent/<owner>/<state_type>/<slug>`

where `<slug>` is the rendering of the non-owner part of the state key; the schema specifies the rendering function.

8.3 Receipt-history enumeration via URA

[normative] Given a state object’s URA, a verifier **MUST** be able to enumerate the full ordered sequence of `CanonicalReceipts` that have advanced the object’s state, up to a chosen `state_version` (or up to the present), without consulting any channel other than the receipt log addressable through that URA.

The receipt-history view **MAY** be exposed as a sub-URA of the form `<state_object_ura>/receipts`, but the URA form is not protocol-fixed; the binding requirement is that the enumeration is reachable through the same trust boundary as the state object itself.

8.4 Append-only

[normative] The receipt log is append-only. Receipts **MUST NOT** be deleted, renumbered, or rewritten; corrections are made by emitting a new receipt that supersedes (but does not erase) the prior one. Implementations that compact, archive, or shard the log **MUST** preserve the externally-observable enumeration contract: a client repeating an enumeration query receives the same sequence (or a strict prefix), modulo new receipts appended since.

9 Invariant Coverage Matrix

This section verifies that the main body anchors every invariant declared in Appendix B (the companion pressure-test) and that every locally-declared invariant in this document is anchored to one main-body section. If any row in either table is missing or inconsistent, the main body is incomplete and must be revised before Phase 1 implementation begins.

9.1 Appendix B invariants → main-body anchors

Invariant	Statement (abridged)	Anchor
TR-INV-1	Long-running transition completion semantics	§4.5
TR-INV-2	Progress receipts not in canonical chain	§4, §4.5
TR-INV-3	Schema-level canonicity marking	§2.7, §3
TR-INV-4	Owner authority over delegated execution	§6.2
TR-INV-5	Runtime overlay observable but not canonical	§4, §5.5
TR-INV-6	View $\neq$ State	§7
TR-INV-7	Multi-view as first-class	§7, §7.3
TR-INV-8	Cross-state-space composition	§5.5
TR-INV-9	Concurrent canonical transitions	§4.7 (RC-INV-2)
TR-INV-10	Content-as-blob boundary	§3.3 (CSH-INV-2)
TR-INV-11	URA-receipt traceability	§8
TR-INV-12	Hub-translated caller identity	§6.4
TR-INV-13	Caller-context as observability channel	§4.6

9.2 Main-body invariants (self-contained)

Invariant	Statement (abridged)	Defined in
SO-INV-1	State-key immutability after first canonical receipt	§2.5
CSH-INV-1	Hash computed over projection, not raw state	§3
CSH-INV-2	Content-as-blob boundary	§3.3
RC-INV-1	Receipt binding to (state_key, transition_id, attempt_id)	§4.4
RC-INV-2	Pre-state-hash declaration and rejection	§4.7
TA-INV-1	Receipt-to-transition binding	§5
VP-INV-1	ability_surface derived from state and schema	§7

9.3 Coverage statement

[[derived](#)] The two tables above account for every TR-INV-* invariant declared in the companion appendix and for every SO/CSH/RC/TA/VP-INV-* invariant declared in this main body. No invariant is left without a normative anchor; no anchor is referenced by zero invariants. The Phase 1 implementation work plan (§10) is therefore well-founded: each work item discharges a specific subset of the rows above.

10 Phase 1 implementation work plan

[[derived](#)] The work this RFC immediately authorises is bounded. Each item below is a Phase 1 deliverable in `EasyNet-Cli`, mapping to one or more invariants from §9.

1. **1a. Schema-level canonical:bool annotation.** Extend `AbilityDescriptor` and state-type schemas with the `canonical` flag; reject schemas missing it. Discharges TR-INV-3, SO-INV-1 (key-field annotation).

2. **1b. Two-class receipt split.** Introduce `CanonicalReceipt` and `OperationalReceipt` types, with the fields of §4; `owner_signature` stub-implemented (signing scheme choice from §6.2). Discharges TR-INV-1, TR-INV-2, TR-INV-5, RC-INV-1, RC-INV-2, TA-INV-1.
3. **1c. principal/ caller URI parser and classifier.** Extend `envelope.caller` parser; add a fail-fast classifier; ensure scope-check distinguishes `agent/` from `principal/`. Discharges TR-INV-12.
4. **1d. Minimal blob store.** A local content-addressed blob store implementing `put/get/exists`; GC and ref-counting stubbed (resolved in RFC-007). Discharges TR-INV-10, CSH-INV-2.
5. **1e. State-key receipt index.** Index the receipt log by state key in addition to receipt id; expose URA-keyed enumeration. Discharges TR-INV-11.

The first reference state machine atop these primitives is `easynet.page` (Appendix A of the companion document, to be drafted). The pressure-test machine is `easynet.web_app` (Appendix B of the companion document, already drafted). View projection (§7) and `caller_context` (§4.6) are exercised end-to-end only when the Hub-class deployment is built; the protocol surface is complete before that engineering work begins.

11 Closing

The intent of this RFC is not to specify every detail of a stateful agent network; it is to specify the smallest surface a conformant implementation must implement before further work can proceed. Where the appendix pressure-tests revealed an invariant, the main body addresses it; where the implementation work plan requires a primitive, the main body defines it. Everything else — formal theorems, hub deployment, web reference machines, cross-binding multi-realm consistency — is deferred to companion artefacts.

Companion artefacts.

- AXON-RFC-006-stateful-easynet.md — Appendix B (`easynet.web_app`) and the thirteen TR-INV-* invariants this main body satisfies.
- Appendix A (`easynet.page`, the simple worked example) — **[deferred]** to a follow-up commit on the companion document.
- RFC-007 — blob store interface and GC rules.
- RFC-008 — delegated execution authorisation tokens and threshold-signature ownership.

Bindingness. Sections marked **[normative]** are binding on every Phase 1 (and later) implementation. Sections marked **[derived]** are negotiable only by showing the derivation incorrect. Sections marked **[deferred]** name the artefact that resolves them; an implementation that needs the resolution sooner than that artefact lands must request an out-of-band exception.

This document is a pre-implementation minimal normative core. It is paired with the companion appendix (Appendix B) which served as its pressure test. If a future revision of either document creates a conflict between them, the appendix wins on matters of invariant coverage and the main body wins on matters of normative form; reconciliation is a separate alignment round.